

electron/prefetch-summaries.cjs · Code explanation (L64-216)

Kind: modified · Baseline: 46a8da0664d1 · Target: NovaDiff · Requested mode: exact · Effective mode: exact ·
Generated: 2026-05-19T23:01:10.871Z

What it does `runSummaryPrefetchLoop` orchestrates a background pre-fetch of per-file summaries for a diff. It receives a configuration object (`opts`) and an `AbortSignal` . It first calls `runCompareEngine` with the `prefetch-summary-queue` command to obtain a plan of files to process, then emits a `summary-prefetch-progress` event with initial statistics (jobs count, skipped counts, eligible changes). For each job it fetches the file diff via `runCompareEngine` (`get-file-diff`), builds diff chunks, constructs a payload for the LLM, calls `summarizeChange` , stores the result in `summaryByPath` , and optionally writes artifacts to `rightRoot` . Progress updates are sent after each file, and errors are reported with specific reasons (e.g., binary/non-text). The loop respects `signal.aborted` to stop early. Finally, it emits a `finished` event indicating completion or abortion.

How it works
1. Plan generation – `runCompareEngine(..., cmd: "prefetch-summary-queue")` returns a plan; the code extracts `queue` , `input_changes` , `skipped_*` , and `eligible_changes` .
2. **Progress init*

- – `webContents?.send("summary-prefetch-progress", { state: "started", ... })` .3. **Job loop*
- – For each `job` in `jobs` : - Abort check (`signal.aborted`) . - Retrieve diff via `runCompareEngine(..., cmd: "get-file-diff")` . - Build `chunks` with `buildDiffExcerptChunks` . - Build `payload` differently if `chunks.length > 1` . - Await `summarizeChange(payload)` and store in `summaryByPath` . - If `rightRoot` is set, call `exportFileSummaryArtifacts` . - Emit `file-done` progress.4. **Error handling*
- – Catches exceptions, distinguishes binary/non-text skips, emits `file-skipped` or `file-error` .5. **Finish*
- – Sends `finished` event with `aborted` flag.

Notes for reviewers- The function relies on external helpers (`runCompareEngine` , `buildDiffExcerptChunks` , `summarizeChange` , `exportFileSummaryArtifacts`) whose behavior is not shown; assume they perform I/O and LLM calls.- Progress events use `webContents?.send` , so if `webContents` is undefined, all notifications are silently dropped.- Skipped counts default to zero if missing from the plan; this could mask unexpected plan shapes.- The loop continues even after a skipped file; only `signal.aborted` stops it.- Error messages are stringified; non-Error

throws may lose stack traces.- `summaryByPath` is a global map; ensure it's cleared or scoped appropriately elsewhere.